



*From Parallel Clusters
to Hyper-distributed applications:*

Programming Swarms with COLMENA

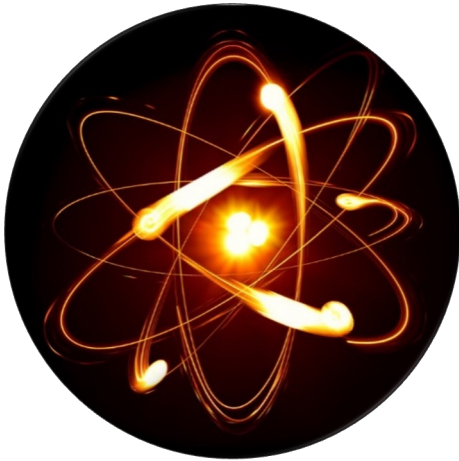
Francesc Lordan

November 16, 2025

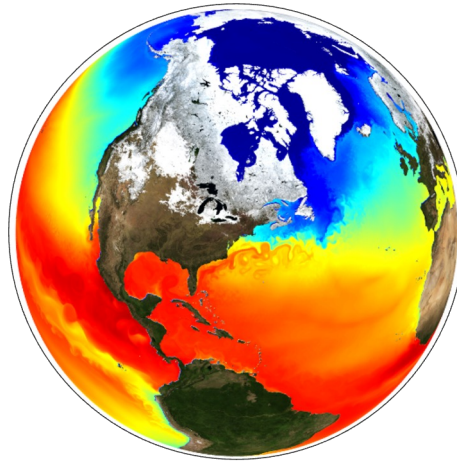
The 8th Annual Parallel Applications Workshop, Alternatives To MPI+X

Science: the Driving Force for Parallel Computing

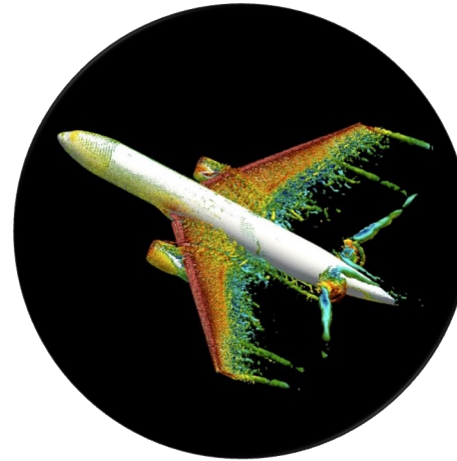
Nuclear Physics



Weather and climate



Fluid dynamics



Astrophysics



Scientific simulations and modeling have an ever-growing need for computing power

Parallel Architectures Inspired by Scientific Workloads

- New parallel architectures emerged as response to this need of computing power



Vector
Computers



Clusters

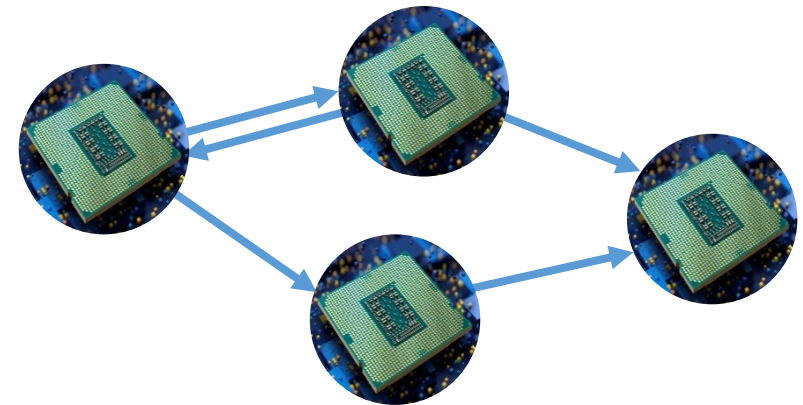
Supercomputers



Programming Parallel Systems

From Manual Effort to MPI

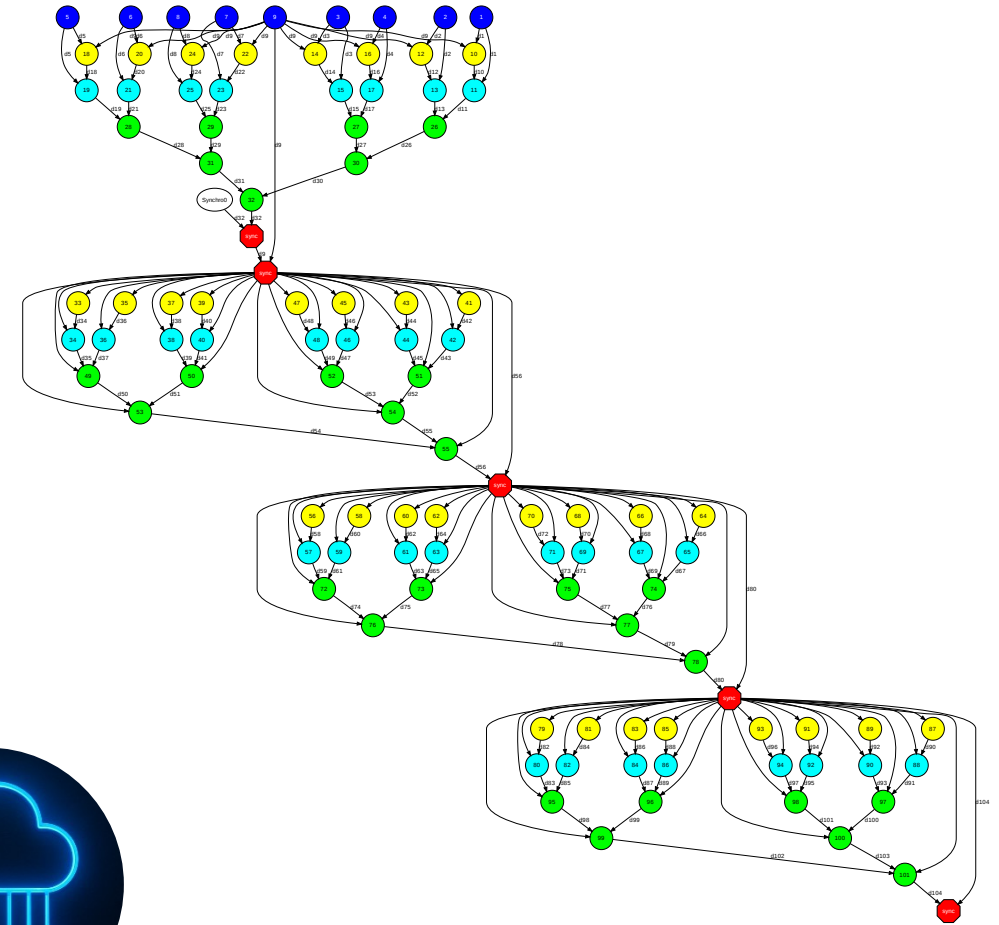
- MPI (Message Passing Interface): standard library for portable parallel programming
 - Abstracts low-level communication
 - Enables: sending/receiving messages, collective operations, synchronization.
- Manual MPI programming requires:
 - Explicit task decomposition
 - Manual data distribution
 - Careful synchronization



Workflow Managers

Abstracting Parallelism and Data Distribution

- Provide a **high-level abstraction** for tasks and data.
 - Often complemented with programming models
- Their runtime systems automatically handle:
 - Task parallelism and dependency management
 - Task scheduling and dispatching
 - Data distribution
- Transparently adapts to any underlying infrastructure
 - Clusters, Grid, Cloud



The new computing landscape:

Global, heterogenous and unreliable

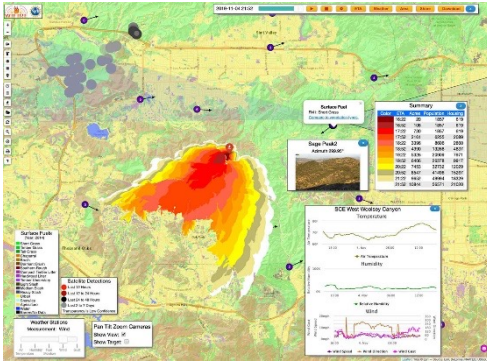
- High-speed mobile and wireless networks enable participation of **computing devices anywhere**
- Characteristics of this new environment:
 - **Heterogeneity**
 - Devices vary widely in compute power, storage, network connectivity, and reliability.
 - **Unpredictability:**
 - Nodes may join or leave dynamically
 - Failures and network fluctuations are common
- Traditional static HPC assumptions no longer hold
 - Workflow Managers evolve to agent-based systems



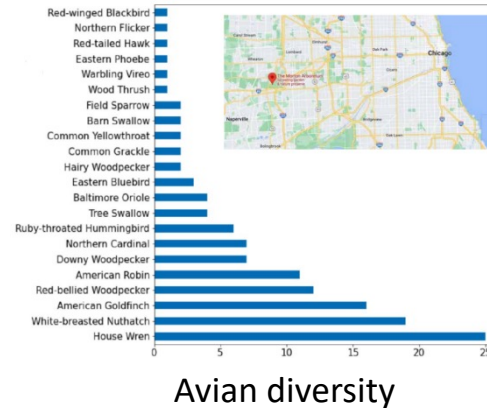
From Simulation to Real-Time Science

Data at Planetary Scale

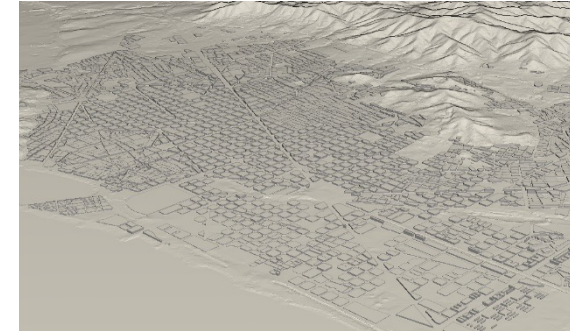
- **A new paradigm for Science:**
 - Scientists are no longer limited to “what-if” simulations and modeling. They can now **observe and analyze the world in real time** through a continuous flow of data.



Wildfire prediction and protection



Avian diversity



Urban-scale atmospheric dispersal

- **Scientific Computing transformation:**
 - From a single workflow executed as batch jobs in HPC clusters
 - To many simultaneous workflows triggered by events on a shared worldwide platform (the Continuum)

Challenges of the Continuum

- No programming environments for developing services as a whole
 - Services are a composition of many sub-services (including communication)
 - Container virtualization technology to encapsulate these sub-services
 - Implementation is driven by deployment simplicity (static)
- Continuum management is heavy and centralized
 - Very large number of devices
 - System with a wide heterogeneity of computing and networking resources
 - Dynamicity of the network and workload
 - The orchestration is Cloud-centric



Colmena

20 BSC **YEARS**

Challenges of the Continuum

- No programming environments for developing services as a whole
 - Services are a composition of many sub-services (including communication)
 - Container virtualization technology to encapsulate these sub-services
 - Implementation is driven by deployment simplicity (static)
- Continuum management is heavy and centralized
 - Very large number of devices
 - System with a wide heterogeneity of computing and networking resources
 - Dynamicity of the network and workload
 - The orchestration is Cloud-centric

COLMENA Programming Model

Ease coding and integration

COLMENA Software Stack

decentralizing the orchestration
building on the autonomy of each
device

Autonomous ageNTs (ANTs)

the foundation of COLMENA

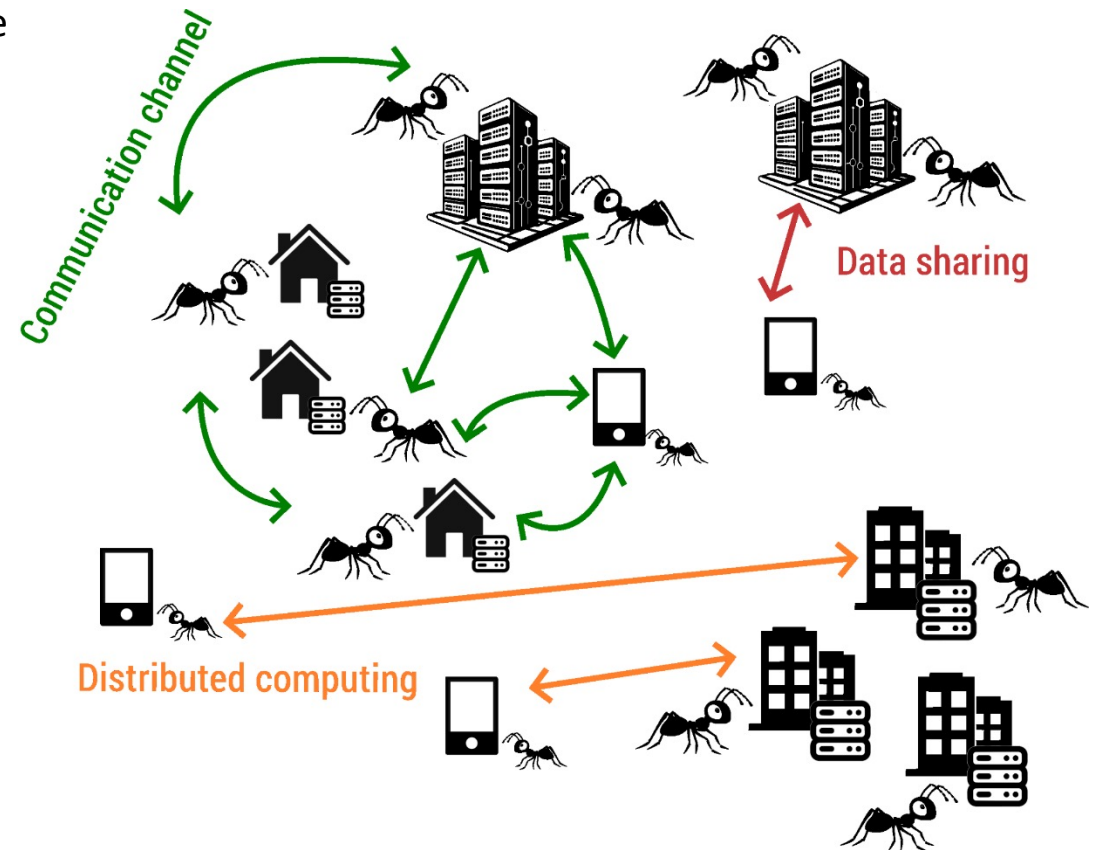
- Every device acts as an Autonomous agent (ANT)
- Each ANT maintains self-awareness of:
 - Skills & capabilities: hardware, performance, power source, ..
 - Context:
 - Local: usage, resource availability, ownership, location ...
 - Social: neighboring agents, their capabilities, ...



Services

Compositions of Interacting Roles

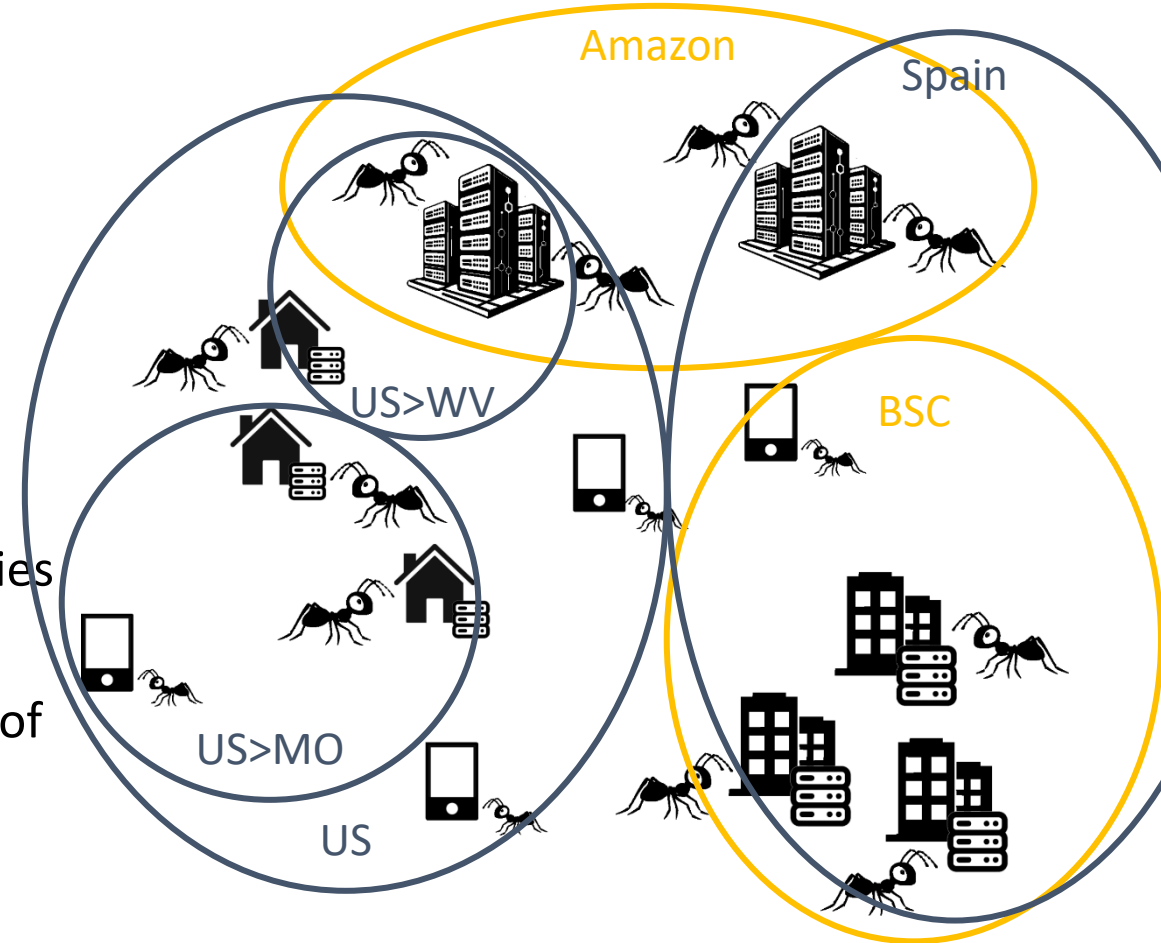
- Service are expressed as a **set of roles**
 - each representing a functional unit or responsibility
 - e.g., sensing, preprocessing, aggregation, model inference
- Roles **interact and collaborate**
 - To achieve a greater purpose
- COLMENA provides built-in mechanisms to :
 - **Communication channels**
 - Pub/sub streams.
 - **Stateful data sharing**
 - Key-Value get/set.
 - **Shared software repositories.**
 - **Workload sharing**
 - Leveraging workflows managers (COMPSSs)



Context

The scoping mechanism in COLMENA

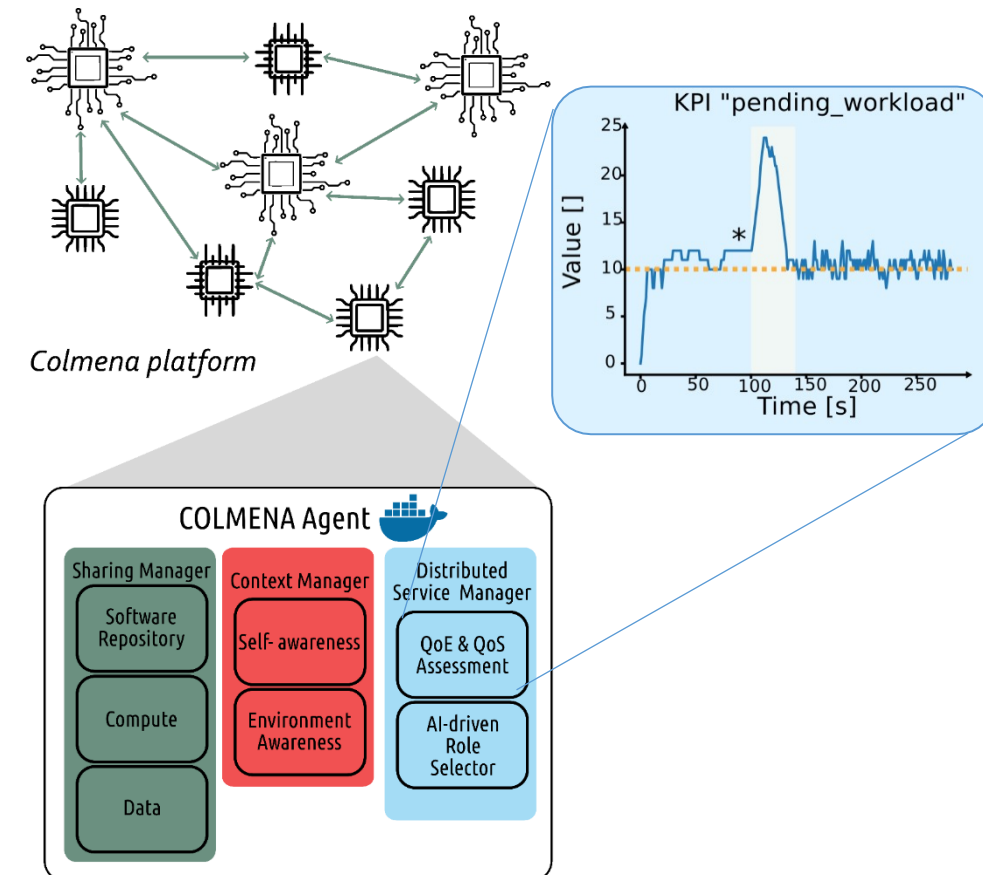
- Context defines circumstantial properties of the devices
- Context is represented by multiple different hierarchies
 - Ownership (Company > Department > Group)
 - Location (Country > State > City)
 - COLMENA provides some built-in context
 - Service developers can create their own context
- ANTs continuously classify themselves in these hierarchies
- Context hierarchies can be combined to limit the scope of
 - Role placement
 - Collaboration mechanisms



Metrics

the driver for autonomous role deployment

- COLMENA runs away from a centralized management
 - relies on the autonomy of each device to decide which roles to execute
- Metrics and KPIs play a key role
 - Developers establish to QoE/QoS objective for each role
 - Each ANT
 - monitors QoS, QoE and performance metrics
 - to evaluate system state and trends
 - Selects role to host and deploys them
- ANTs can also coordinate for decision-making
 - e.g., Consensus
- Context can limit the scope of KPIs to trigger a local response



COLMENA

Programming Model

```
class ServiceA(Service):
    @Context(class_ref=Premises, name="premises")
    @Channel(name="buffer", scope=" ")
    @Channel(name="result", scope=" ")
    @Metric(name="sensed")
    @Metric(name="processed")
    def __init__(self, *args, **kwargs):
        # ...
```

```
class Premises(Context):
    def __init__(self, *args, **kwargs):
        self.structure = {"floor1": ["reception"]}
        # ...

    def locate(self, image):
        # ...
```

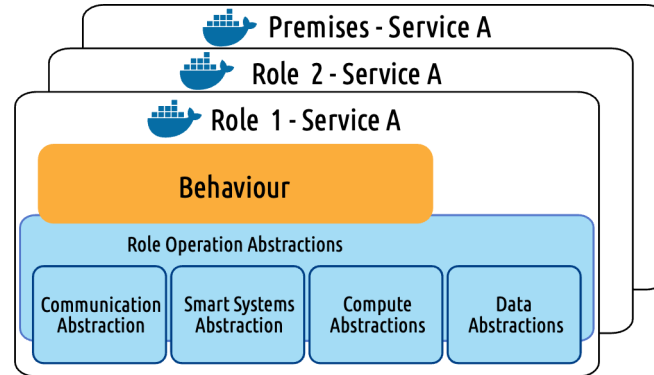
```
class Role2(Role):
```

```
class Role1(Role):
    @Channel(name="result")
    @Channel(name="buffer")
    @Metric(name="processed")
    @Requirements("GPU")
    @KPI("buffer_queue_size < 10")
    def __init__(self, *args, **kwargs):
        # ...

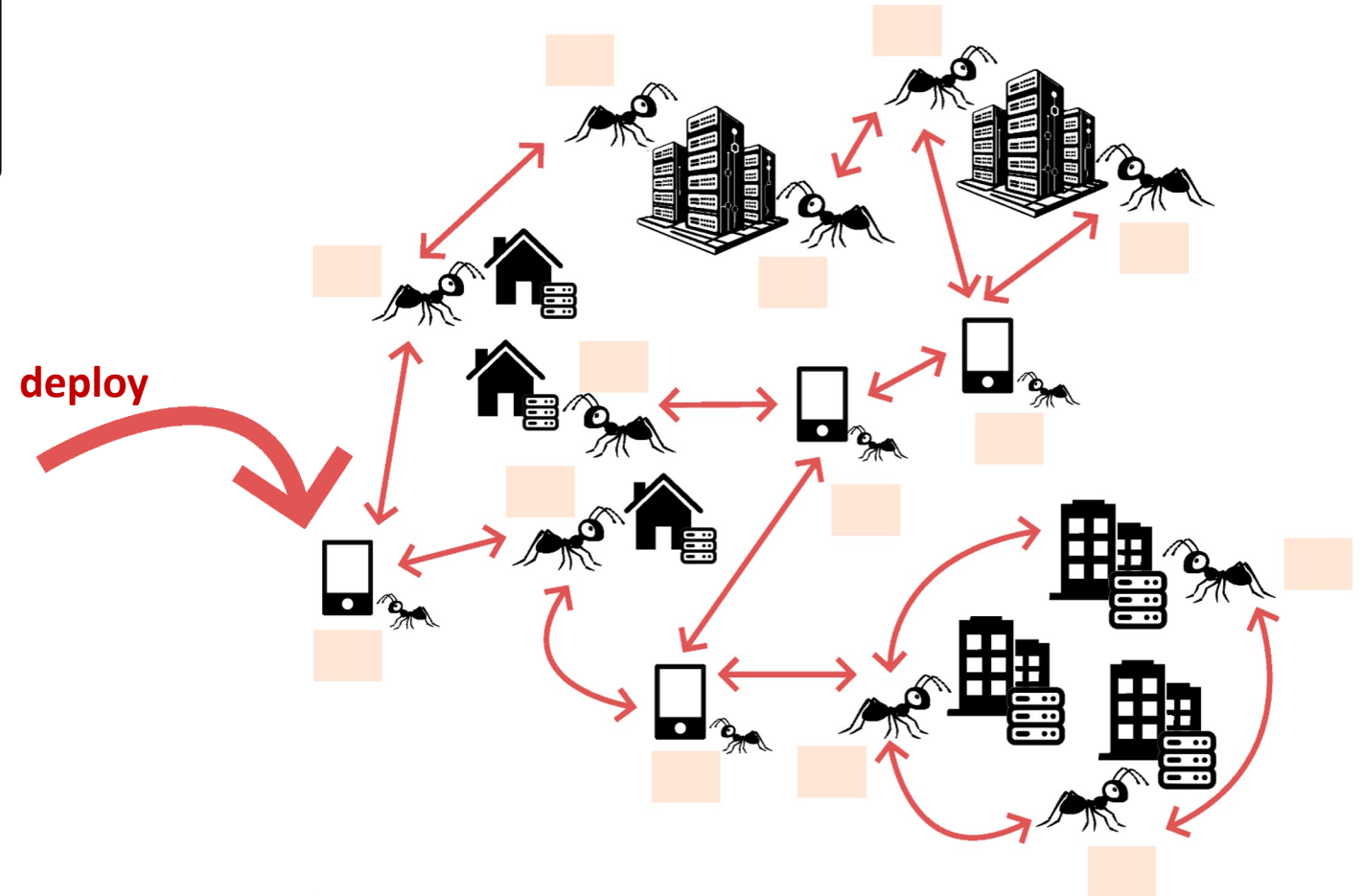
    @Async(image="buffer")
    def behavior(self, image):
        # ...
```

COLMENA

Deployment



```
service_model.json
{
  "id": {
    "value": "ServiceA"
  },
  "dockerContextDefinitions": [
    {
      "id": "premises",
      "imageId": "colmena/premises"
    }
  ],
  "kpis": [],
  "dockerRoleDefinitions": [
    {
      "id": "Role1",
      "imageId": "colmena/rolex",
      "hardwareRequirements": [
        "CPU"
      ],
      "kpis": []
    }
  ]
}
```



Conclusions

- COLMENA presents a novel orchestration approach for the Digital Continuum
- The programming model simplifies the development of services
 - Services are decomposed as several functionalities (roles)
 - Provides several abstraction (e.g., channels) to simplify the integration
- COLMENA converts all the devices into autonomous agents
 - Handling the device performance details automatically
 - Providing tools to present the whole infrastructure as a platform
 - Orchestration is fully distributed, therefore avoiding the Cloud

20^{BSC} YEARS



Colmena



<https://proyecto-colmena.com/>



<https://github.com/colmena-swarm>

francesc.lordan@bsc.es

Software Stack

Layer 4:
Service

Layer 3:
Service Tools

Swarm
Intelligence
Library

Swarm
Programming
Model

Role
Operation
Abstraction

Service
Deployment
Tool

Service
Operation
Dashboard

Layer 2:
Swarm Platform

Sharing
Manager

Software
Repository

Data

Compute

Distributed
Service
Manager

QoE & QoS
Assessment

AI-driven
Role
Selector

Context Manager

Self
Awareness

Environment
Awareness

Trust

Layer 1:
ANT Handler

Communication
Controller

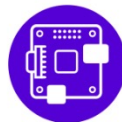
Smart System
Controller

ANT
Monitor

Role
Optimiser

ANT
Configuration
Manager

Layer 0:
Hardware Node
Virtual Instance
Virtual Platform



Network
Interfaces



Sensors &
Actuators

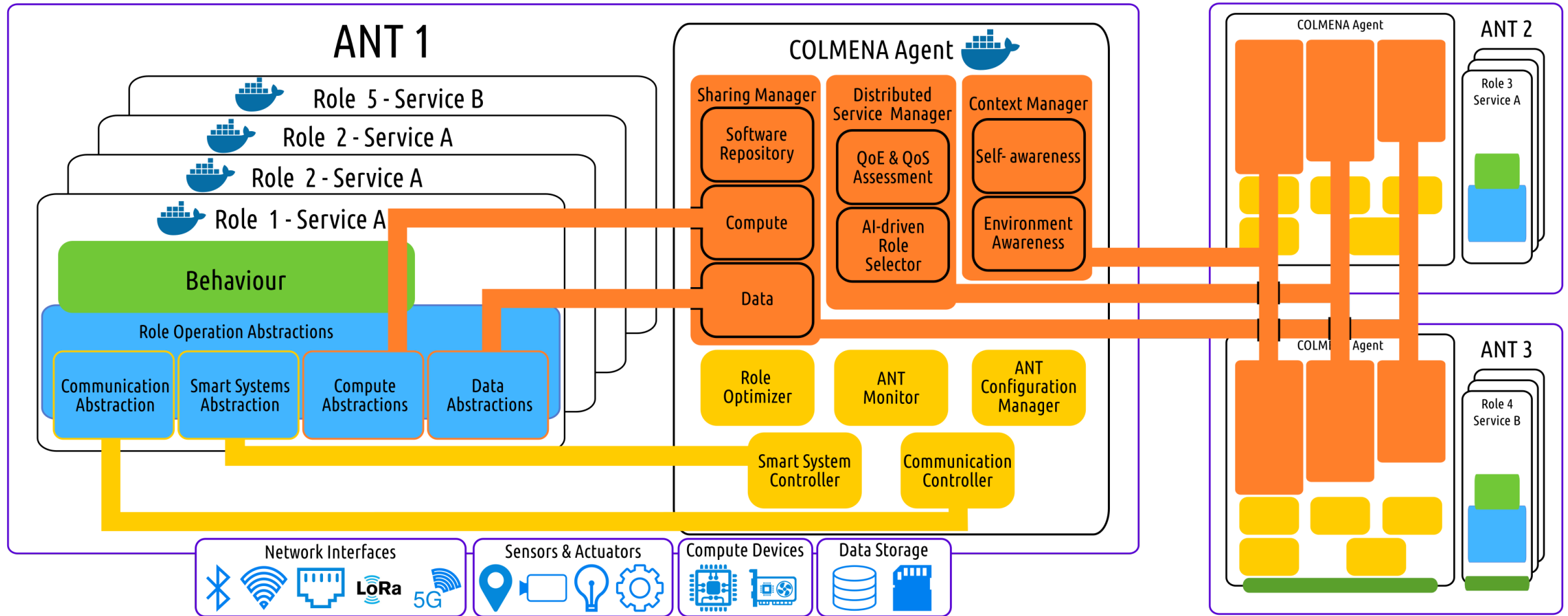


Compute
Devices



Data
Storage

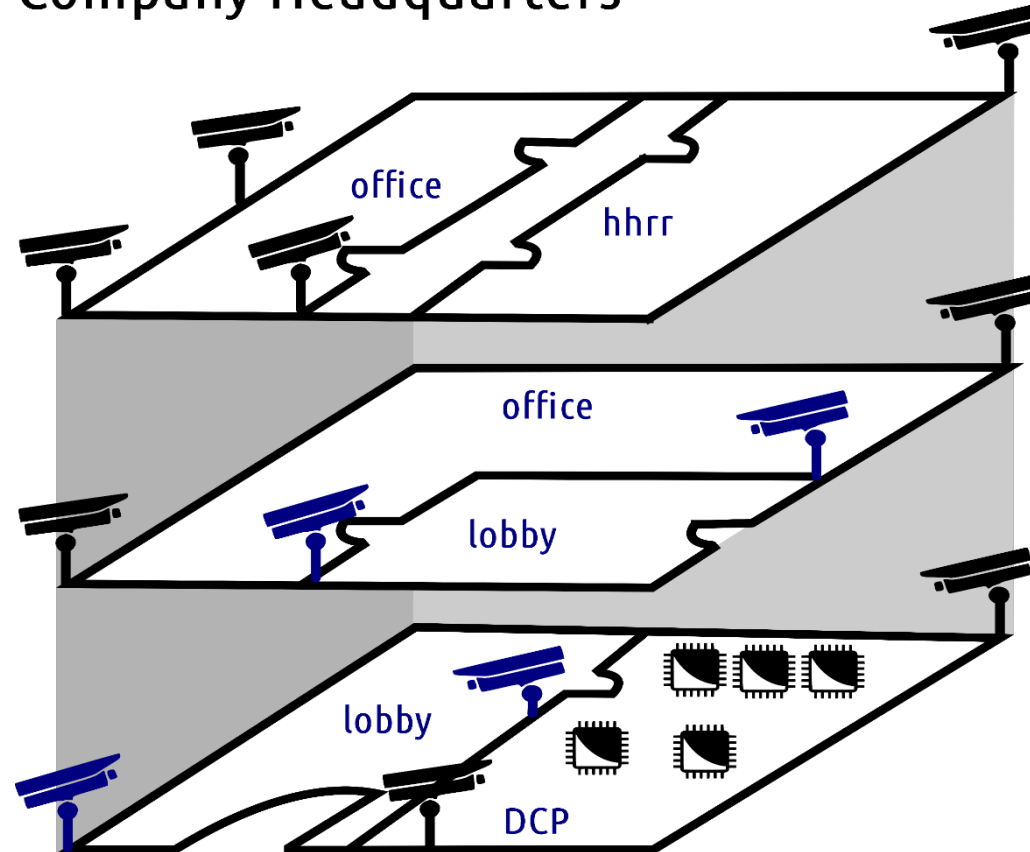
Component deployment



Pilot use case

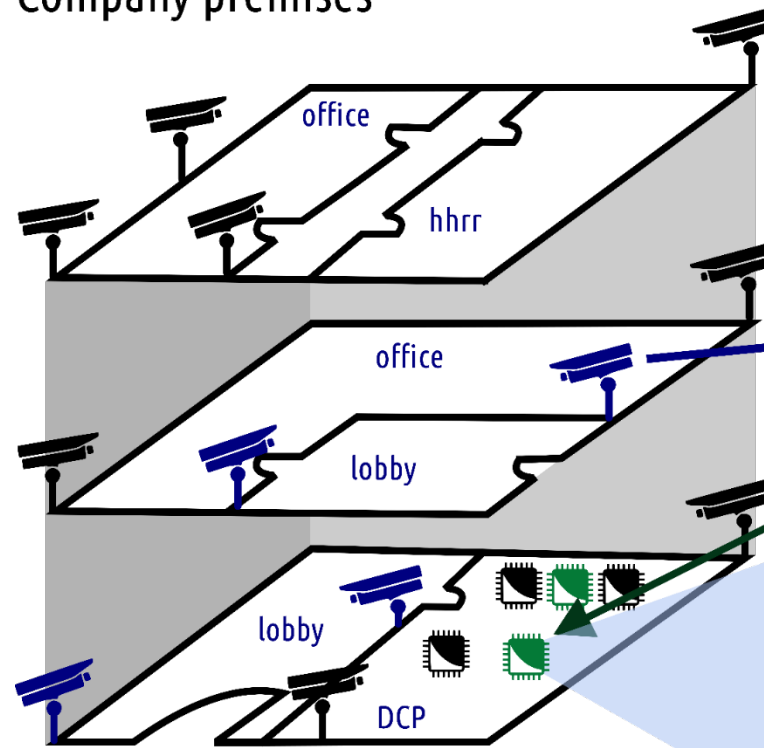
Video Surveillance Service processing all the images from cameras in the lobbies using a computer vision software

Company Headquarters



Pilot deployment example

Company premises



Roles

-  Sensing
-  Processing

Requirements:

- /devices/camera
- /processor/CPU

Role abstractions

Communication
and Sharing

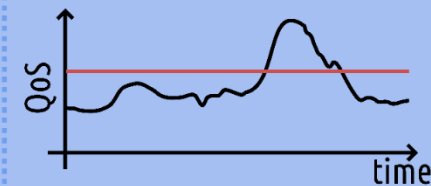
Metric ("processing_time")

Channel ("pending_images")

COLMENA ANT

QoS assessment

$\text{avg}(\text{processing_time})[2s] < 3s$



Programming model example

- 2 roles
 - **Sensing**: collect images
 - **Processing**: process images with CV
- 1 cooperation mechanism
 - **pending_images** (Channel)
- 1 custom metric
 - **processing_time**

```
class VideoSurveillance(Service):
```

```
    @Context("premises", class=Premises)
    @Channel("pending_images", scope="")
    @Metric("processing_time")
    def __init__(self, *args, **kwargs):
        pass
```

```
class Sensing(Role):
```

```
    @Requirements("/devices/camera")
    @Context("premises", scope="*/lobby")
    @Channel("pending_images")
    @KPI("num_players = 1")
    def __init__(self, *arg, **kwarg):
        # Initialize the role
```

```
    @Persistent()
```

```
    def behavior(self):
        image = # Get image from camera
        time = time.time()
        self.pending_images.publish([time, image])
```

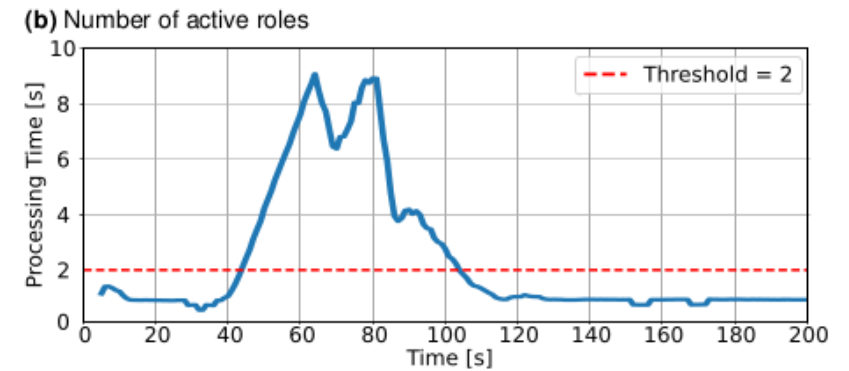
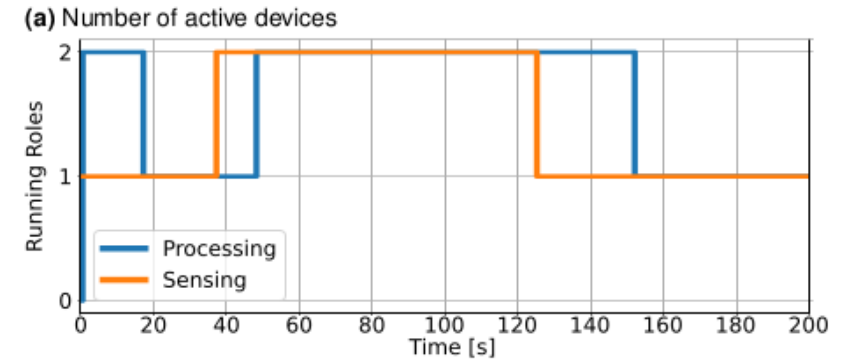
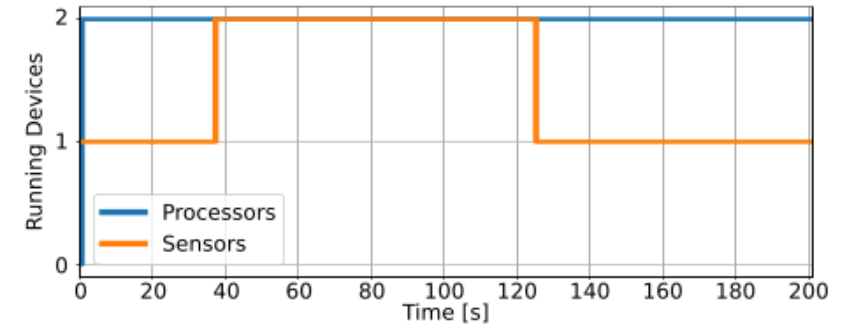
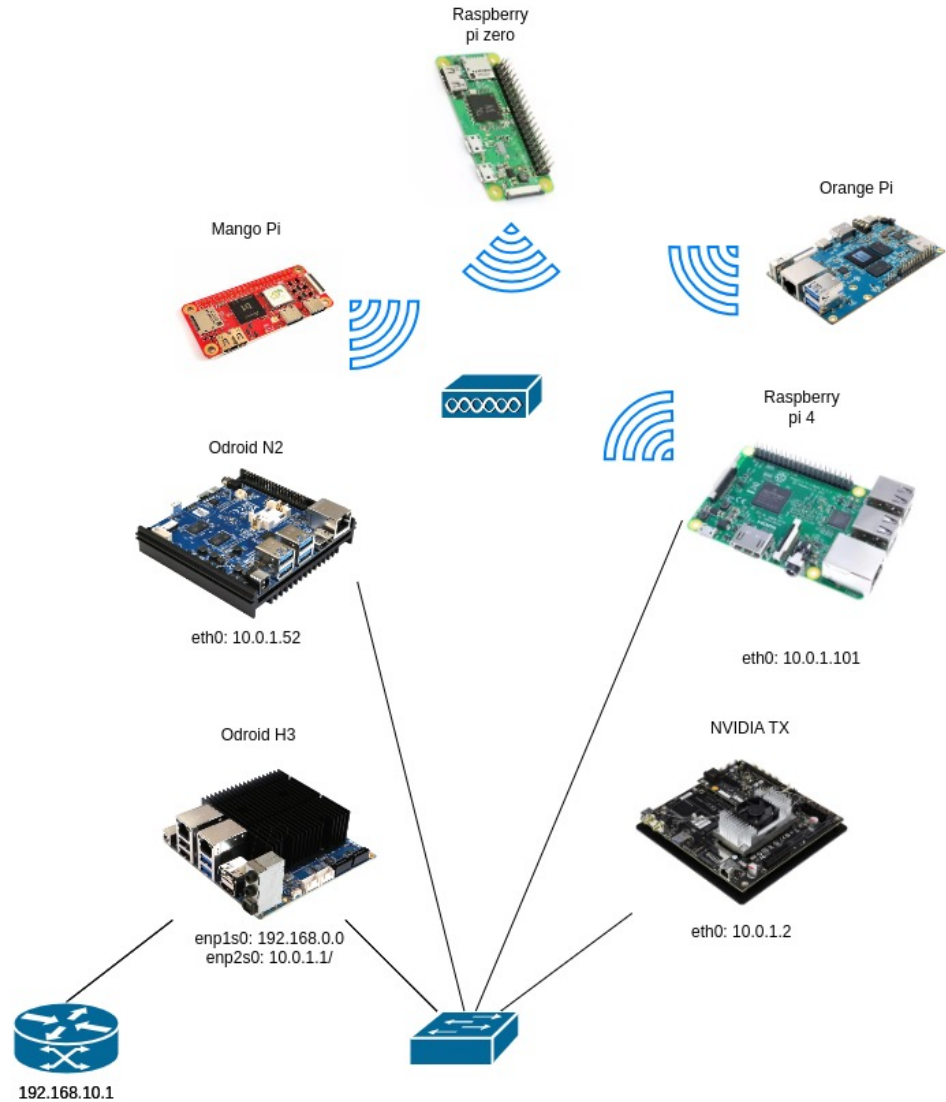
```
class Processing(Role):
```

```
    @Requirements("/processor/GPU")
    @Channel("pending_images")
    @Metric("processing_time")
    @KPI("processing_time[2s] < 3")
    def __init__(self, *arg, **kwarg):
        # Initialize the role
```

```
    @Async(message="pending_images")
```

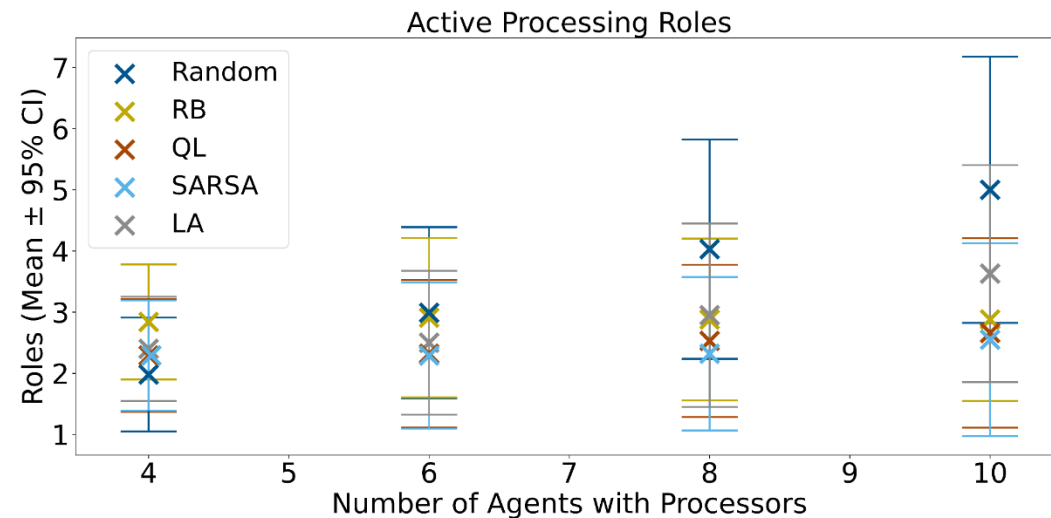
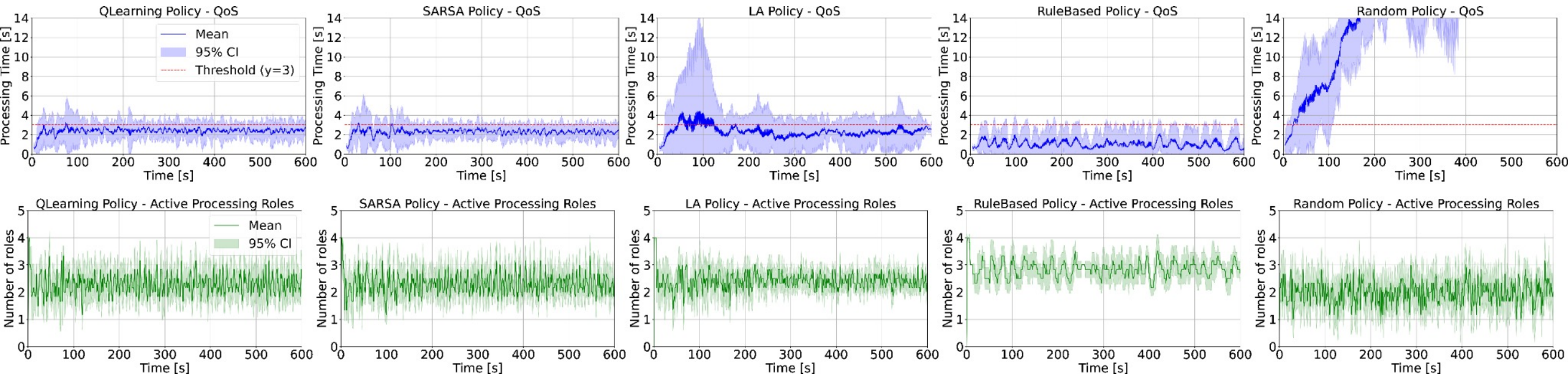
```
    def behavior(self, message):
        t_capture = message[0]
        image = message[1]
        process_image(image)
        t_processed = time.time()
        delay = t_processed - t_capture
        self.processing_time.publish(delay)
```


Deployment on testbed

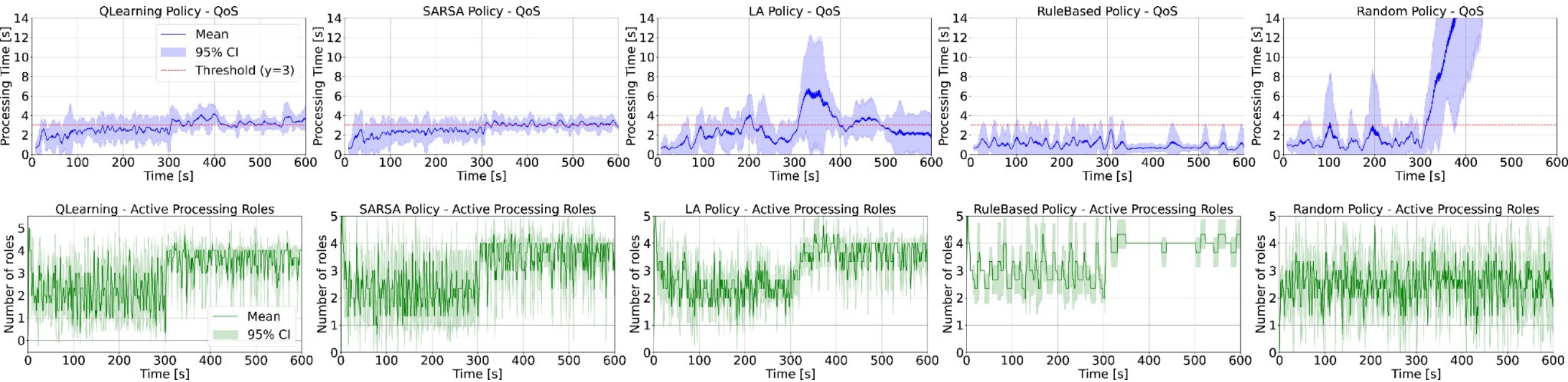


(c) Processing time KPI

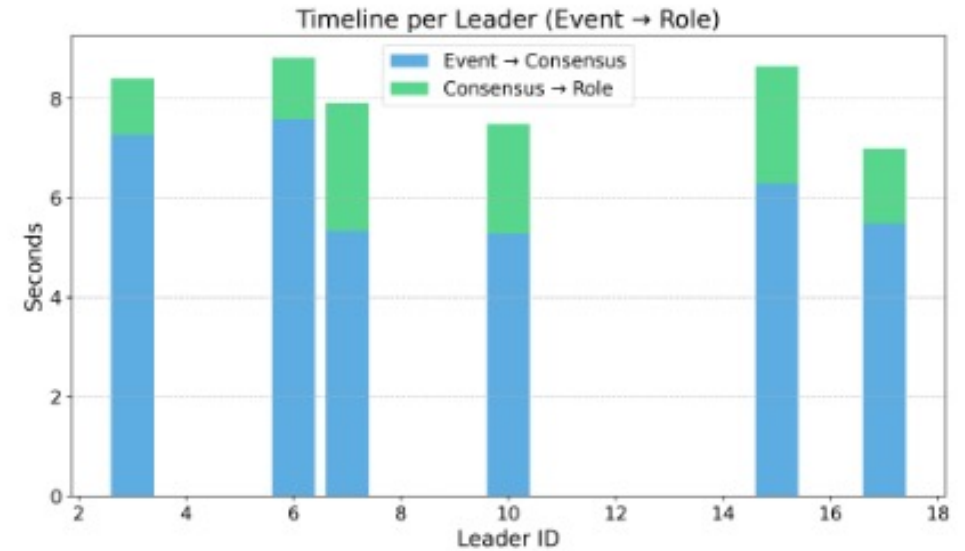
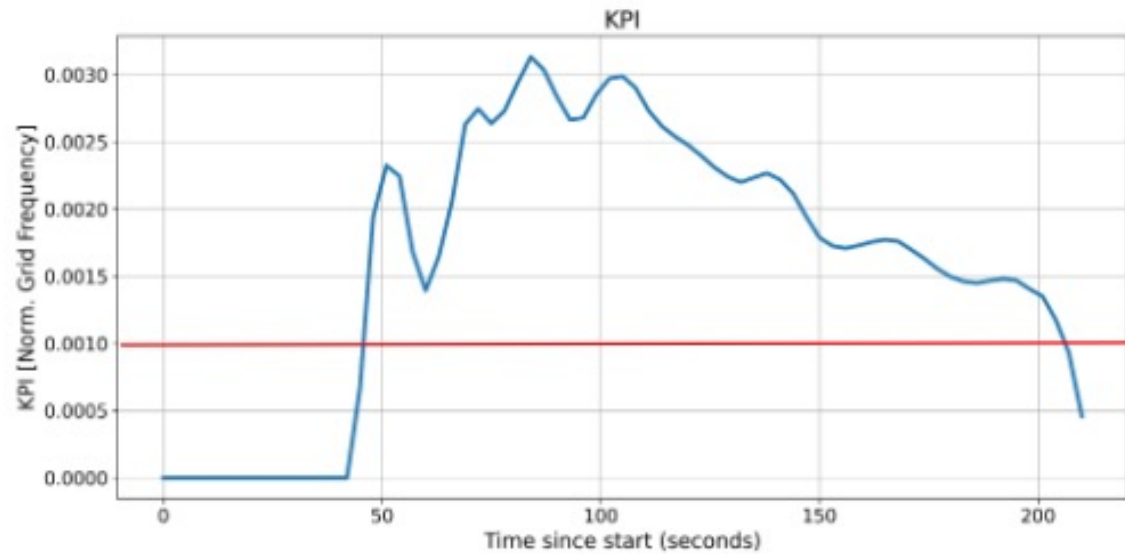
Policy evaluation



Policy Evaluation – dynamic infra



Preliminary results



	leader_id	dt_event_to_consensus	dt_consensus_to_role	dt_role_to_kpi	area
1	3	7.27	1.14	157.74	1
2	6	7.58	1.25	157.32	2
3	7	5.34	2.57	159.18	3
4	10	5.28	2.21	159.68	4
5	15	6.28	2.37	159.21	5
6	17	5.48	1.51	160.24	6

